

AVP Seminar 1주차

학습 계획 및 수행 내용

최민규

Architecture and Compiler for Embedded System LAB.

School of Electronics Engineering, KNU, KOREA

2026-05-26



KNU

목차

- ROS2 학습 계획
- simulation 과제 수행
- 인지/학습 파트 수행

ROS2 학습 계획

- 핑크랩 PinkLAB 유튜브 활용
- 무작정 따라하는 ROS2
(입문, 응용, 실전, 심화 과정)

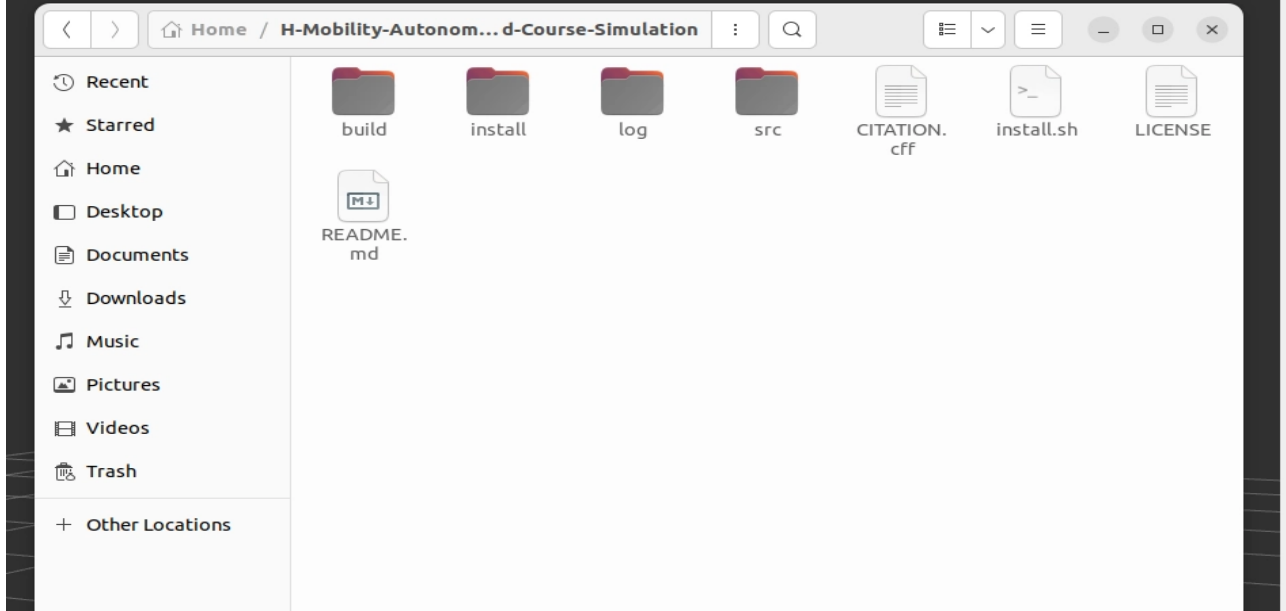


1주차	ROS2 개념 및 인지/학습 파트 공부
2주차	무작정 따라하는 ROS2 : 입문 과정 (기본 명령어, 서비스, Topic, Action, 클래스 등)
3주차	무작정 따라하는 ROS2 : 응용 과정1 (package와 node, 서비스 및 액션 서버 만들기 등) -> 1 ~ 3-10 수강
4주차	무작정 따라하는 ROS2 : 응용 과정2 (파라미터 다루기, LOG, RQT, rosbag 등) -> 4-1 ~ 5-11 수강
5주차	무작정 따라하는 ROS2 : 실전 과정1 (OpenCV, Optical Flow node 등)
6주차	무작정 따라하는 ROS2 : 실전 과정2 (ROS_DOMAIN_ID, PID 제어 활용 turtlesim, statemachine으로 주행 제어기 구현)
7주차	무작정 따라하는 ROS2 : 심화 과정1 (gazebo 활용 및 실습)
8주차	무작정 따라하는 ROS2 : 심화 과정2 (SLAM, AMCL, Nav2 등)

simulation 구현 (1)

• 기본 구성 파일

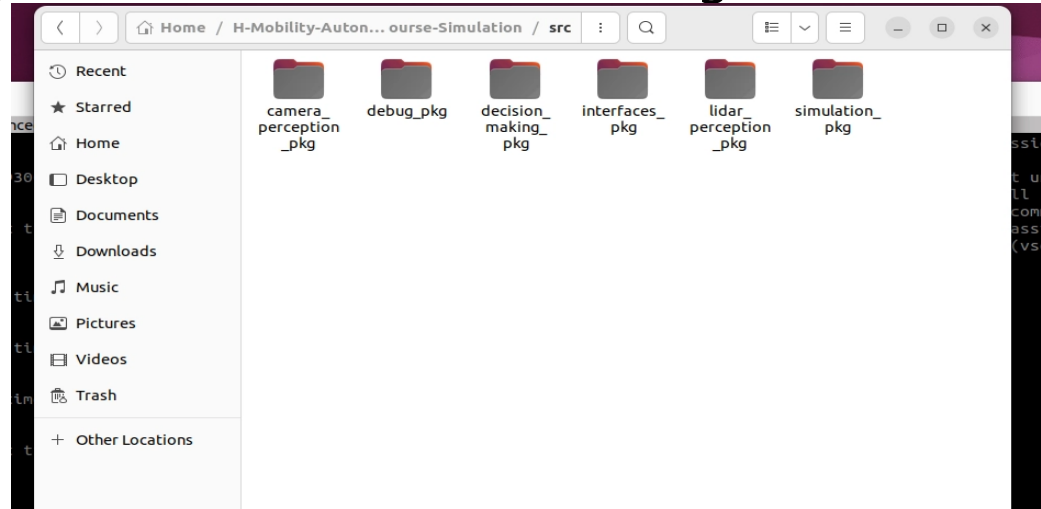
- src : 직접 작성한 소스 코드들이 들어있는 폴더
- build : 컴퓨터가 src에 있는 코드를 읽고, 기계어로 컴파일하는 중간 작업 공간
- install : 빌드가 성공적으로 끝나고 만들어진 최종 실행파일이 세팅된 폴더
- log : 실행되는 동안 발생한 모든 기록(경고, 에러, 성공여부)이 txt파일로 저장



simulation 구현 (2)

- src 파일 내부 구성

- **camera_perception_pkg & lidar_perception_pkg** : 차량의 눈 역할
카메라로 차선을 찾고, 라이다나 YOLO로 장애물 인식
- **debug_pkg** : 모든 과정이 잘 되고 있는지 Rviz 화면을 띄워주는 모니터링
- **decision_making_pkg** : 인식 데이터를 바탕으로 경로(Path)와 Motion 제어
- **interfaces_pkg** : 노드들끼리 주고받을 메시지 형식 (ex: 조향각은 float, 속도 int)
- **simulation_pkg** : 제어 명령을 실제 자동차(gazebo) 장치에 전달하여 구동



simula

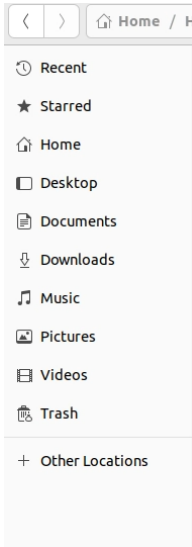
- simulati

simulation

-> 모든 노
스트 가능

```
3 import os
4 import subprocess
5 from launch_ros.actions import Node
6 from launch import LaunchDescription
7 from launch.actions import ExecuteProcess
8 from ament_index_python.packages import get_package_share_directory
9
10 def generate_launch_description():
11
12     subprocess.run(['killall', 'gzserver'])
13     subprocess.run(['killall', 'gzclient'])
14
15     package_dir=get_package_share_directory('simulation_pkg')
16     world_file = os.path.join(package_dir, 'worlds', 'track.world')
17
18     return LaunchDescription([
19         ExecuteProcess(
20             cmd=['gazebo', '--verbose', world_file, '-s', 'libgazebo_ros_factory.so'],
21             output='screen'),
22
23         ExecuteProcess(
24             cmd=['rqt'],
25             output='screen'),
26
27         Node(
28             package='simulation_pkg',
29             executable='video_recording_node',
30             output='screen'
31         ),
32
33         Node(
34             package='simulation_pkg',
35             executable='load_ego_car_node',
36             output='screen'
37         ),
38
39         #Node(
40         #     package='camera_perception_pkg',
41         #     executable='yolov8_node',
42         #     output='screen'
43         #),
44
45         #Node(
46         #     package='debug_pkg',
47         #     executable='yolov8_visualizer_node',
48         #     output='screen'
49         #),
50
51         Node(
52             package='debug_pkg',
53             executable='path_visualizer_node',
54             output='screen'
55         ),
56
57         Node(
58             package='camera_perception_pkg',
59             executable='lane_info_extractor_node',
60             output='screen'
61         ),
62
63         #Node(
64         #     package='decision_making_pkg',
65         #     executable='path_planner_node',
66         #     output='screen'
67         #),
68
69         #Node(
70         #     package='decision_making_pkg',
71         #     executable='motion_planner_node',
72         #     output='screen'
73         #),
74
75         #Node(
76         #     package='simulation_pkg',
77         #     executable='sim_simulation_sender_node',
78         #     output='screen'
79         #),
80
81     ])
```

성능 테



g_sim.launch.py" selected (2.1 kB)



simulation 구현 (4)

- camera_perception_pkg/lane_info_extractor_node.py

```
self.subscriber = self.create_subscription(DetectionArray, self.sub_topic, self.yolov8_detections_callback, self.qos_profile)

self.publisher = self.create_publisher(LaneInfo, self.pub_topic, self.qos_profile)

# ROI 이미지 퍼블리셔 추가
self.roi_image_publisher = self.create_publisher(Image, ROI_IMAGE_TOPIC_NAME, self.qos_profile)
```

[YOLOv8 노드] --DetectionArray--> [이 노드] --LaneInfo--> [제어 노드]

--Image-----> [시각화/디버깅]

-> YOLOv8 객체 탐지 결과를 받아 차선 정보를 계산하고 ROI 이미지를 함께 발행하는 구조

- **subscriber(구독자)** : YOLOv8 노드가 발행하는 DetectionArray(탐지된 객체들의 배열) 메시지가 도착할 때마다 yolov8_detections_callback 함수가 자동으로 호출된다.

- **publisher(발행자)** : 차선 정보(LaneInfo)를 외부로 내보내는 발행자로, 코드에서 self.publisher.publish(메시지)를 명시적으로 호출해야 발행됨 (보통 콜백함수의 마지막에 호출)

- **roi_image_publisher** : 차선 인식에 사용한 관심영역(Region of Interest) 이미지를 발행

> **실시각화**로 어디를 보는지, ROI가 제대로 잡혔는지 rqt image view나 RViz2로 시각화

simulation 구현 (5)

• yolov8_detections_callback 함수

핵심 데이터 처리 📷 카메라 원근 뷰 (Camera Perspective View)

새로운 Detector
자동으로 실행되

- if len(detection

- lane2_edge_im

(detection_msg,

- (h, w) = (lane2.

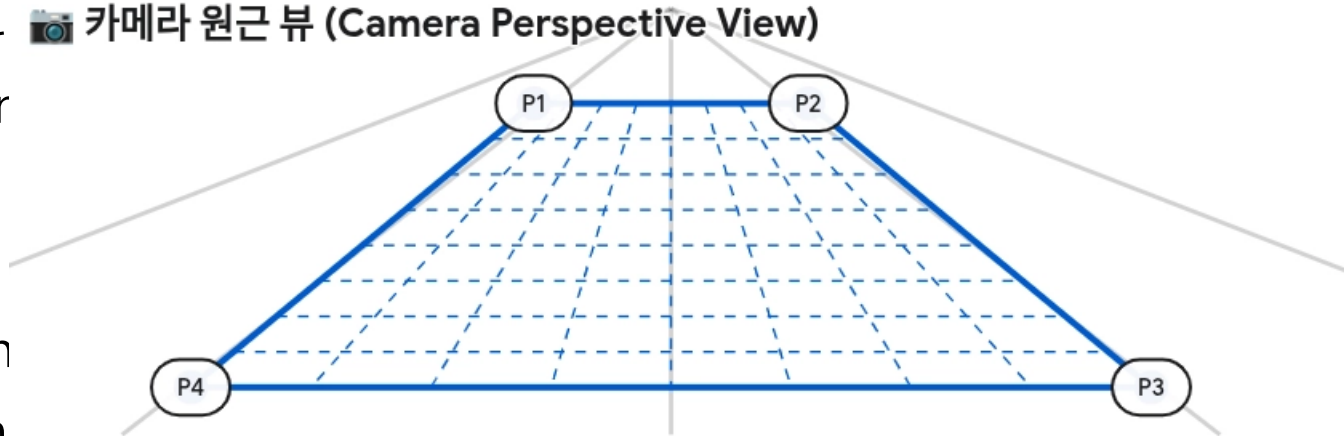
lane2_edge_ima

- grad = CPFL.d

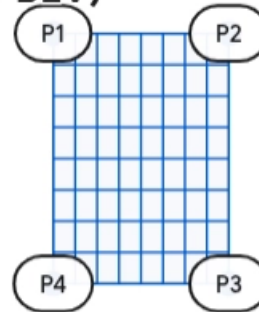
theta_limit=70)

- target_points = []

- self.publisher.publish(lane)



🦅 조감도 뷰 (Bird-Eye View / BEV)



```
def yolov8_detections_callback(self, detection_msg: DetectionArray):  
    if len(detection_msg.detections) == 0:  
        return  
  
    lane2_edge_image = CPFL.draw_edges(detection_msg, cls_name="lane2", color=255)
```

```
shape[1]) #(480, 640)  
round(h * 0.0)], [round(w * 0.7),
```

```
srcmat=src_mat, dstmat=dst,  
cutting_idx=300)
```

int8로 변환

ge, encoding="mono8")

image: {e}")

자 50씩 증가

tion_height=target_point_y,
nt=grad, lane_width=300)

```
lane = LaneInfo()  
lane.slope = grad  
lane.target_points = target_points
```

```
self.publisher.publish(lane)
```

simulation 구현 (6)

- camera_perception_pkg/yolov8_node
 - YOLO란 ?

You Only Look Once, 이름 그대로 이미지를 딱 한 번만 쳐다보고 그 안에 어떤 물체들이 어디에 있는지 즉각적으로 찾아내는 인공지능 모델

- YOLO 이전의 객체 탐지 인공지능(ex: R-CNN)은 2단계로 작동
 1. 이 부분 뭔가 물체가 있을 것 같은데 ? 하고 이미지에서 수천 개의 후보 영역을 먼저 잘라낸다(Region Proposal)
 2. 잘라낸 영역들을 하나씩 인공지능에 다시 집어넣어 자동차인지, 사람인지 분류한다.
- 정확도는 높았지만 연산량이 너무 많아 자율주행처럼 1초에 30프레임 이상을 처리해야 하는 환경에서는 속도가 너무 느렸다.

simulation 구현 (7)



simulation 구현 (8)

- YOLOv8Node

- LifecycleNode

설정(configure) → 활성화(Activate) →

비활성화(Deactivate) → 종료(Shutdown)

-> YOLO는 무거운 작업이므로 준비가 끝날 때까지 기다렸다가 안전하게 시작 (준비되기 전 동작X)

- #self.declare_parameter("model", "yolov8m.pt")
self.declare_parameter("model", "sim.pt")

- self.declare_parameter("device", "cpu")
#self.declare_parameter("device", "cuda:0")

- self.declare_parameter("threshold", 0.5)

```
class YOLOv8Node(LifecycleNode):
```

```
def __init__(self, **kwargs) -> None:
```

```
    super().__init__("yolov8_node", **kwargs)
```

```
#-----Variable Setting-----
```

```
# 딥러닝 모델 pt 파일명 작성
```

```
#self.declare_parameter("model", "yolov8m.pt")
```

```
self.declare_parameter("model", "sim.pt")
```

```
# 추론 하드웨어 선택 (cpu / gpu)
```

```
self.declare_parameter("device", "cpu")
```

```
#self.declare_parameter("device", "cuda:0")
```

```
#-----
```

```
self.declare_parameter("threshold", 0.5)
```

```
self.declare_parameter("enable", True)
```

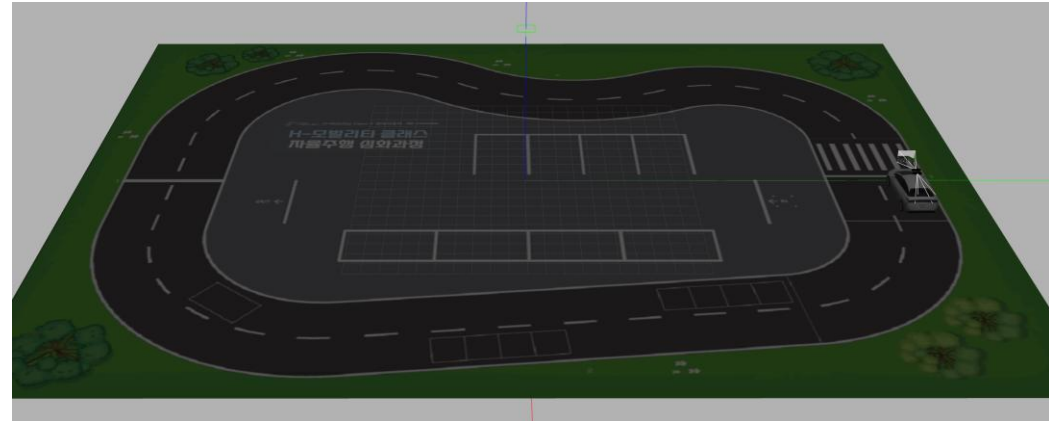
```
self.declare_parameter("image_reliability",  
                        QoSReliabilityPolicy.RELIABLE)
```

```
self.get_logger().info("YOLOv8Node created")
```



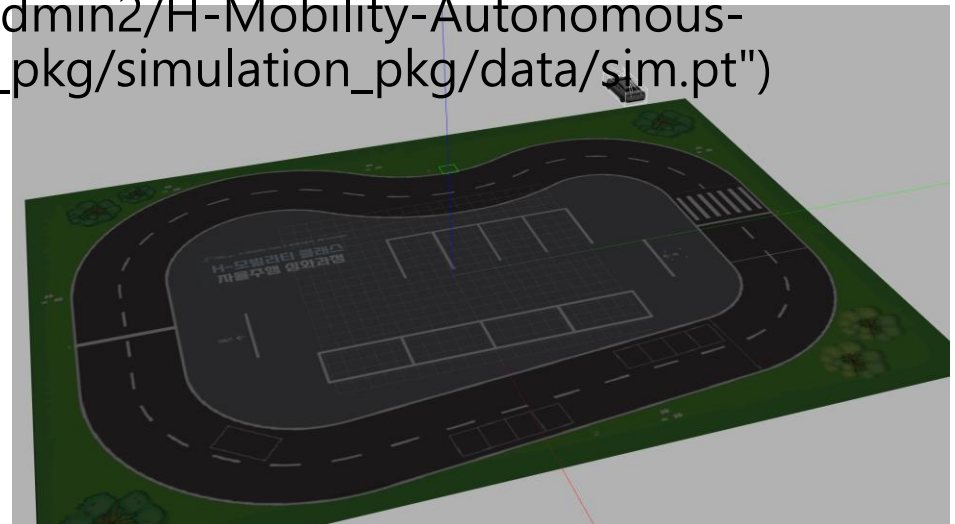
simulation 구현(9)

- 차가 가만히 있는 오류
-> 코드 내 유령 공백문제 해결



- 차가 차선을 인식하지 못해 맵을 벗어나는 오류
-> `self.declare_parameter("sim.pt")`으로 지정했더니 `sim.pt`의 경로를 찾지 못해 `sim.pt` 파일을 찾지 못함
→ `self.declare_parameter("model", "/home/admin2/H-Mobility-Autonomous-Advanced-Course-Simulation/src/simulation_pkg/simulation_pkg/data/sim.pt")`

- 차가 차선을 자꾸 침범함
-> 스티어링을 3,-3으로 낮추어서 구현



인지/학습 파트 - 학습 모델 제작 (1)

- 라벨링 파일 500장을 통해 학습 모델(.pt) 만들기

1. 학습 환경 세팅 (Ultralytics 설치)
2. 데이터셋 디렉토리 구조 맞추기 (8:2, train(학습용):val(검증용))
3. data.yaml 파일 생성

yaml 파일 : 설정(setting)을 사람이 읽기 쉽게 적어놓는 텍스트 파일 (.yaml or .yml)

4. 학습

```
admin2@admin: ~/track_train/dataset 118x27
2/150 3.25G 0.9466 2.098 1.401 1.406 0 44 640: 52%
2/150 3.25G 0.9462 2.084 1.39 1.4 0 56 640: 56%
2/150 3.25G 0.9519 2.082 1.389 1.407 0 38 640: 60%
2/150 3.25G 0.9525 2.09 1.385 1.409 0 40 640: 64%
2/150 3.25G 0.9563 2.113 1.391 1.405 0 47 640: 68%
2/150 3.25G 0.9583 2.098 1.385 1.405 0 50 640: 72%
2/150 3.25G 0.9581 2.079 1.382 1.405 0 42 640: 76%
2/150 3.25G 0.9622 2.05 1.379 1.407 0 49 640: 80%
2/150 3.25G 0.959 2.033 1.373 1.402 0 38 640: 84%
2/150 3.25G 0.9578 2.031 1.371 1.399 0 41 640: 88%
2/150 3.25G 0.9492 2.011 1.366 1.393 0 39 640: 92%
2/150 3.25G 0.9476 2 1.361 1.389 0 52 640: 96%
2/150 3.25G 0.9476 2 1.361 1.389 0 52 640: 100%
/25 8.5it/s 3.0s
Class Images Instances Box(P R mAP50 mAP50-95 Mask(P R mA
Class Images Instances Box(P R mAP50 mAP50-95 Mask(P R mA
mAP50-95): 100% 2/2 7.1it/s 0.3s
all 50 50 0.793 0.6 0.804 0.368 0.453 0.331 0.
0.0812
Epoch GPU_mem box_loss seg_loss cls_loss dfl_loss sem_loss Instances Size
3/150 3.25G 1.012 1.845 1.38 1.44 0 41 640: 0%
3/150 3.25G 0.9277 1.671 1.353 1.372 0 40 640: 4%
3/150 3.25G 0.9355 1.674 1.337 1.385 0 44 640: 8%
Class Images Instances Box(P R mAP50 mAP50-95 Mask(P R mA
Class Images Instances Box(P R mAP50 mAP50-95 Mask(P R mA
P50 mAP50-95): 100% 2/2 10.4it/s 0.2s
all 50 50 0.999 1 0.995 0.989 0.999 1 0.
995 0.987
```

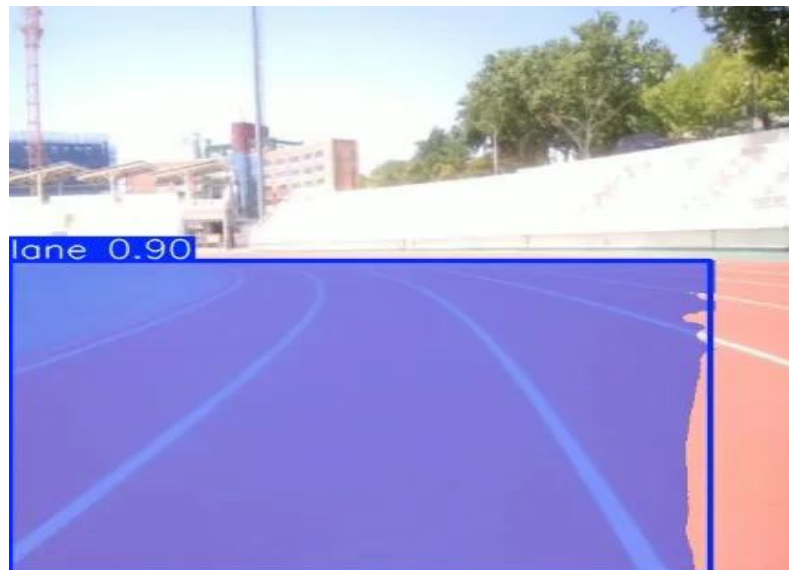


인지/학습 파트 - 학습 모델 제작 (2)

- 라벨링 포맷 오류

- YOLOv8 Detection 포맷

- > 축에 정렬된 사각형(bounding box) 만 표현

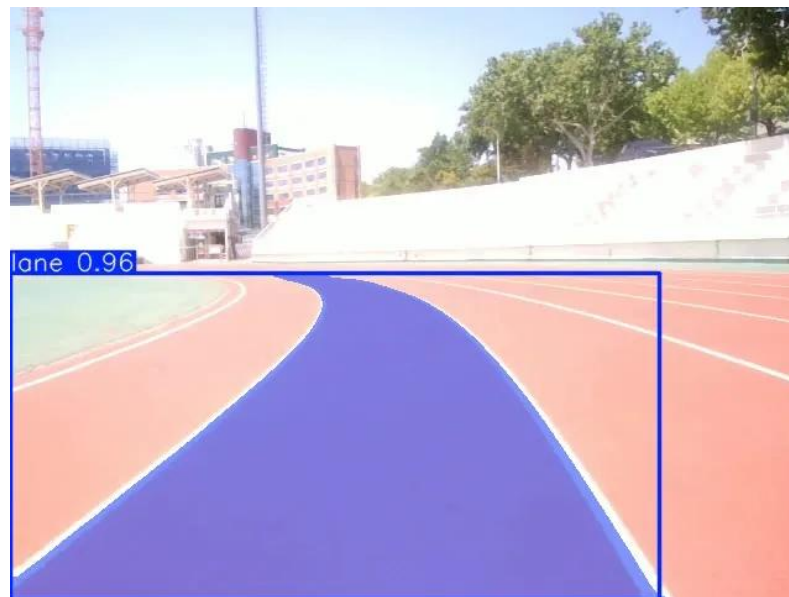


- YOLOv8 (Segmentation) 포맷

1. Bounding Box (사각형 외곽)

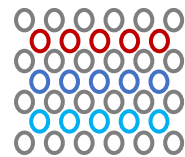
2. Segmentation Mask (정확한 영역)

둘 다 표현



Q & A

Thank you for your attention



Architecture and Compiler
for Embedded Systems Lab.

Graduate School of Electronic and Electrical Engineering, KNU
ACE Lab. (hw051656@gmail.com)